Università degli Studi di Napoli – Federico II

Progetto di macchine sequenziali

richiami e introduzione alla progettazione VHDL

ARCHITETTURA *dei* **SISTEMI DIGITALI**

Prof.ssa Alessandra De Benedictis

Sistemi sequenziali

- Le **macchine sequenziali** o, più in generale, i **sistemi sequenziali**, sono sistemi in cui le uscite in un certo istante dipendono non solo dal valore degli ingressi nello stesso istante, ma anche dagli ingressi precedenti
  - Le informazioni sull'attività passata del sistema ne costituiscono lo **stato**, che informalmente può essere definito come la ***memoria interna*** di un sistema

- Nell'ambito dei sistemi digitali, oltre che per la realizzazione di dispositivi di memorizzazione, le macchine sequenziali hanno larga applicazione nell'implementazione delle **unità di controllo**
  - L'unità di controllo di un sistema è la parte del sistema che ne definisce il comportamento (ad esempio implementando uno specifico algoritmo), impartendo opportuni comandi (segnali di controllo) alla cosiddetta **parte operativa**, costituita dall'insieme dei componenti architetturali che implementano praticamente le operazioni richieste (es. registri, unità aritmetiche, bus, ecc.)

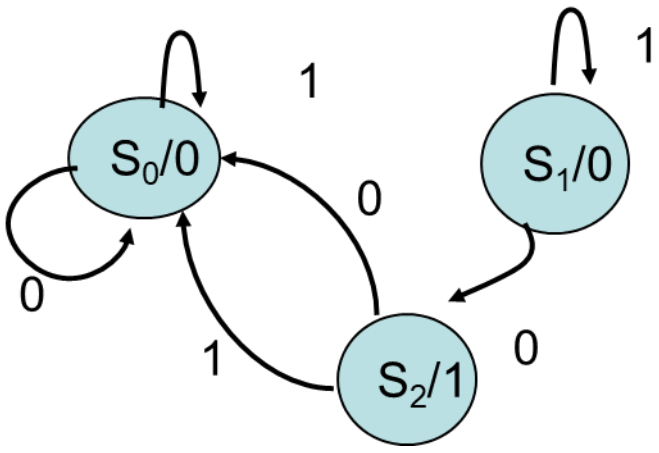
Macchine sequenziali e Automi a Stati Finiti

- Il comportamento di una macchina sequenziale può essere rappresentato in maniera non ambigua tramite un **automa a stati finiti** (ASF), un modello matematico di "sistema" caratterizzato da un insieme finito di ingressi, uscite e stati.

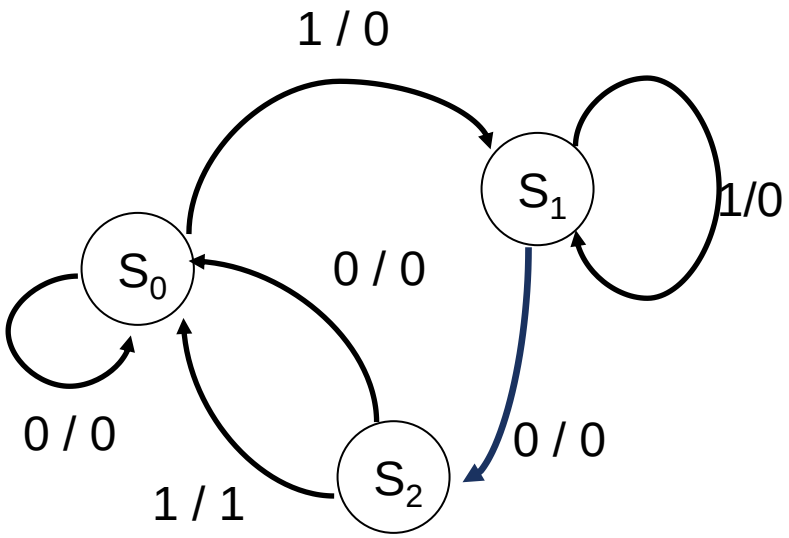
- Un automa è una quintupla <Q, I, U, t, w> dove:
  - Q: insieme finito di ***stati*** interni $q \in Q$
  - I: insieme finito di ***ingressi*** $i \in I$
  - U: insieme finito di ***uscite*** $u \in U$
  - t: **funzione di transizione** (stato prossimo) t: $Q x I \rightarrow Q$
  - w**: funzione di uscita**
    - ❑ w: $Q x I \rightarrow U$ MODELLO DI **MEALY**
    - ❑ w: $Q \rightarrow U$ MODELLO DI **MOORE**

Rappresentazione di un automa mediante grafo

Gli automi sono tipicamente rappresentati mediante **grafi** in cui i nodi rappresentano gli stati e gli archi le transizioni fra stati dipendenti dagli ingressi. Le uscite sono associate allo stato o alla transizione a seconda del modello adottato (Mealy vs Moore).



Moore



Mealy

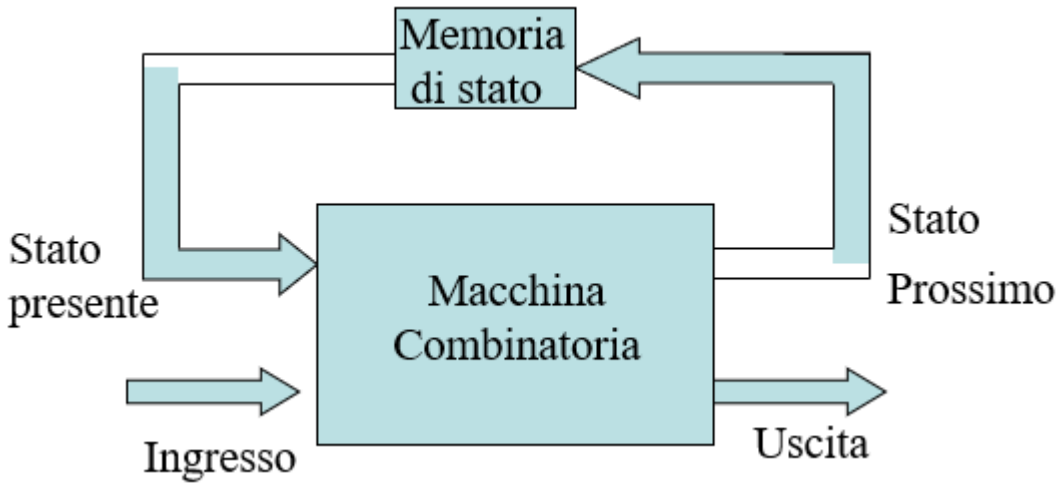
Macchine sincrone e asincrone

- **Macchine Asincrone**: sono macchine che, partendo da qualsiasi stato e applicando un qualsiasi ingresso (anche per un tempo molto lungo), raggiungono sempre uno stato cosiddetto *stabile*, in cui cioè permangono fino all'applicazione di un nuovo ingresso

- **Macchine Sincrone**: sono macchine in cui avviene che almeno per uno stato e per un ingresso, a partire da tale stato e applicando tale ingresso esse continuano la loro evoluzione fra gli stati finché l'ingresso è applicato, senza fermarsi in uno stato stabile.
  - Nella macchine sincrone lo stato in cui si trova la macchina in un dato istante dipende dalla durata del segnale di ingresso e dai tempi di reazione della macchina.

Modello fondamentale di macchina sequenziale

- Il modello di Huffman separa esplicitamente la memoria di stato dalla parte del sistema che calcola le funzioni di transizione e di uscita, che può essere modellata come una macchina combinatoria



Nelle macchine **asincrone**, grazie alle loro caratteristiche intrinseche di stabilità, la retroazione dello stato può essere applicata mediante delle semplici linee.
Nelle macchine **sincrone,** invece, lo stato deve essere memorizzato in una memoria (implementata mediante specifiche macchine sequenziali, i flip-flop), a cui deve essere applicata una apposita tempificazione per evitare che lo stato retroazionato si ricombini con gli ingressi in maniera non corretta causando evoluzioni incontrollate.

Implementazione di macchine sequenziali in VHDL

- Come abbiamo visto per le macchine combinatorie, anche per quelle sequenziali ci sono diversi approcci alla progettazione VHDL
  - **Approccio comportamentale**: viene descritto il comportamento della macchina come definito dal relativo *automa*/diagramma degli stati (**finite state machine**)
  - **Approccio strutturale**: vengono descritti i componenti della macchina e le relative interconnessioni

Descrizione strutturale di una macchina sequenziale sincrona

- Da un punto di vista strutturale, il progetto prevede la descrizione dei componenti della rete logica (porte logiche e flip-flop) e delle loro interconnessioni
- E’ facile notare che, per reti complesse, il numero di componenti da istanziare e di segnali da utilizzare per le interconnessioni potrebbe diventare molto elevato, e dunque la descrizione comportamentale viene solitamente preferita perché più semplice

Descrizione di una FSM in VHDL

- Il costrutto fondamentale per descrivere una FSM è il ***process***

- Il modello più vicino alla realizzazione hardware di una FSM prevede **due processi,** uno puramente combinatorio che calcola la funzione stato prossimo e la funzione di uscita, e l'altro che rappresenta l'elemento di memoria (cioè il banco di flip-flop) che consente di retroazionare lo stato (ed eventualmente di registrare le uscite).
  - E' possibile anche usare un terzo processo per definire il comportamento del segnale di uscita separatamente

- In alternativa, è possibile modellare un automa con **un unico processo** (spesso la descrizione risulta più immediata)

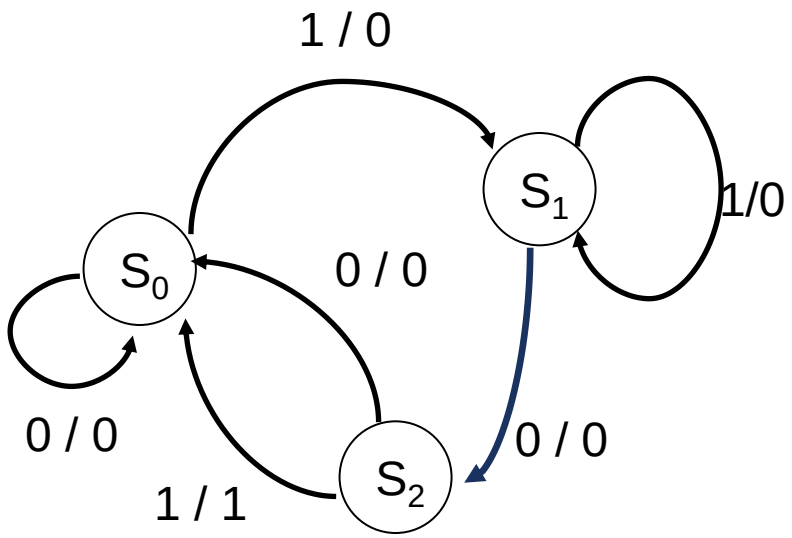
Descrizione comportamentale di una macchina sequenziale sincrona (1/3)

```
entity automa is
    port( i: in std_logic;
          CLK: in std_logic;
          Y: out std_logic
    );
end automa;

architecture Behavioral of automa is

type stato is (S0, S1, S2);
signal stato_corrente : stato := S0;
signal stato_prossimo : stato;

begin

...
```

Per prima cosa definiamo l'interfaccia: questa macchina ha un ingresso binario i e un'uscita binaria Y e prende in ingresso un segnale di tempificazione CLK

Tipicamente per lo stato viene definito un **tipo enumerativo** che contiene tutti i possibili valori.
In alternativa, si potrebbero usare anche il tipo integer, bit_vector o std_logic_vector per dichiarare il registro di stato

Nella parte di dichiarazione dei segnali, dichiariamo i segnali stato_corrente e stato_prossimo con i quali descriveremo la transizione di stato e indichiamo lo stato inziale, assunto all'inizio del runtime dalla macchina

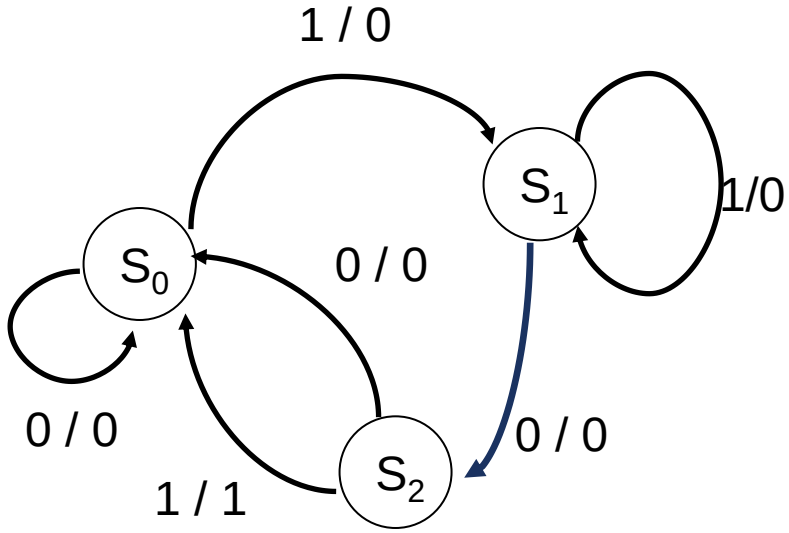


Descrizione comportamentale di una macchina sequenziale sincrona (2/3)

```
f_stato_uscita: process(stato_corrente,i)
    begin
    case stato_corrente is
        when S0 =>
            if( i = '0' ) then
                stato_prossimo <= S0;
                Y <= '0';
            else
                stato_prossimo <= S1;
                Y <= '0';
            end if;
        when S1 =>
            if( i = '0' ) then
                stato_prossimo <= S2;
                Y <= '0';
            else
                stato_prossimo <= S1;
                Y <= '0';
            end if;
        when S2 =>
            if( i = '0' ) then
                stato_prossimo <= S0;
                Y <= '0';
            else
                stato_prossimo <= S0;
                Y <= '1';
            end if;
        when others =>
                stato_prossimo <= S0;
                Y <= '0';
    end case;
end process;
```

Definiamo il processo combinatorio che implementa la funzione di uscita e di stato prossimo.
NOTA: perché sia combinatorio, il processo deve includere nella sensitivity list tutti i segnali letti!

Con il costrutto case…when è possibile definire cosa fa la macchina in ogni stato: nel caso di una macchina di Mealy come quella dell'esempio, ogni valore dell'ingresso fa evolvere la macchina verso un nuovo stato e produce una specifica uscita
NOTA: lo stato prossimo cambia appena cambia l'ingresso i, ma si rifletterà sullo stato corrente solo in determinati istanti



Descrizione comportamentale di una macchina sequenziale sincrona (3/3)

```
mem: process (CLK)
    begin
        if( CLK'event and CLK = '1' ) then
            stato_corrente <= stato_prossimo;
        end if;
end process;
end Behavioral;
```

Definiamo il processo sequenziale che realizza la memoria di stato e consente di retroazionare lo stato prossimo nello stato corrente.
Il process è sensibile al solo segnale di clock.

La memorizzazione avviene sul fronte di salita del clock, che può essere rilevato utilizzando l'attributo **'event** oppure usando il costrutto **rising_edge(clock).**
NOTA: 'event ha effetto ogni volta che c'è una variazione di un segnale e quindi questa condizione risulterebbe vera anche quando ad esempio il segnale passa da 'U' a '1', mentre **rising_edge** si riferisce ad una transizione '0'->'1'.
Questa differenza potrebbe essere in rilevante in simulazione, visto che ogni segnale può assumere 9 valori diversi, mentre non è apprezzabile con la sintesi su FPGA, dove ci sono solo i valori 0 e 1

Descrizione con un processo

```
type stato is (S0, S1, S2);
signal stato_corrente : stato := S0;
begin
stato_uscita_mem: process(CLK)
    begin
    if( CLK'event and CLK = '1' ) then
        case stato_corrente is
            when S0 =>
                if( i = '0' ) then
                    stato_corrente <= S0;
                    Y <= '0';
                else
                    stato_corrente <= S1;
                    Y <= '0';
                end if;
            when S1 =>
                if( i = '0' ) then
                    stato_corrente <= S2;
                    Y <= '0';
                else
                    stato_corrente <= S1;
                    Y <= '0';
                end if;
            when S2 =>
                if( i = '0' ) then
                    stato_corrente <= S0;
                    Y <= '0';
                else
                    stato_corrente <= S0;
                    Y <= '1';
                end if;
            when others =>
                    stato_corrente <= S0;
                    Y <= '0';
        end case;
        end if;
end process;
```

La descrizione con un solo processo utilizza un process sensibile al solo clock, che descrive come deve evolvere la macchina solo in corrispondenza del fronte di salita.
NOTA: Non si utilizzano più due segnali per lo stato corrente e lo stato prossimo, ma un unico segnale che viene modificato direttamente.

Descrizione di un automa nell'ottica della sintesi con i tool Xilinx

- I tool di sintesi incorporati negli IDE di sviluppo della Xilinx sono in grado di identificare propriamente una FSM e quindi di sintetizzarla purché vengano adottati degli accorgimenti nella programmazione:
  - È necessario identificare uno stato iniziale, che la macchina assume al power up
  - È necessario inserire un segnale di RESET, sincrono o asincrono

```
SYNC_PROC : process (clk)
begin
      if rising_edge(clk) then
            if (reset = '1') then
                  state <= S0;
            else
                  state <= next state;
            end if;
      end if;
end process;
```

sincrono

```
process (clk, rst)
begin
    if (rst='1') then
      ...
    elsif (clk'event and clk='1') then
      ...
    end if;
end process;
```

asincrono

  - Se si utilizza la codifica con doppio process è necessario inserire nella sensitivity list del processo combinatorio (funzione stato prossimo e uscita) il segnale di stato (corrente) e tutti gli input della macchina
  - Se l'output deve essere inserito in un registro allora l'assegnazione va fatta nel processo sequenziale (quello che si occupa della retroazione)

Codifica degli stati

- Xilinx supporta diverse tecniche di **codifica degli stati** e può operare una ottimizzazione per ridurre l'area occupata o ottenere prestazioni migliori
  - Auto State Encoding: XST cerca di trovare l'algoritmo più efficace in maniera automatica
  - One-Hot State Encoding (default): associa un bit ad ogni stato e quindi in ogni istante solo una variabile di stato è asserita (appropriato per gli FPGA che hanno a disposizione molti flip-flop)
  - Gray State Encoding: garantisce che cambi un solo bit alla volta quando si passa da uno stato all'altro (indicato quando si implementano controller con molti stati senza salti)
  - Compact State Encoding: minimizza il numero di bit della codifica
  - User State Encoding: usa la codifica specificata dall'utente
  - …

```
type count_state is (zero, one, two, three, four, five, six, seven);
signal my_state : count_state;
attribute fsm_encoding : string;
attribute fsm_encoding of my_state : signal is "sequential";
```

I valori possibili sono: OFF, ONE_HOT, SEQUENTIAL, GRAY, JOHNSON, AUTO